

1. What is a Stream?

A Stream in Java is a sequence of elements that supports functional-style operations on collections of objects. It's not a data structure but a pipeline of computations.

Key Characteristics:

- Not a data structure - Doesn't store data
 - Functional - Uses lambda expressions
 - Lazy evaluation - Operations are not executed until needed
 - Pipeline - Chain of operations (filter, map, etc.)
 - Stateless - Operations don't modify the source
 - Once consumed, cannot be reused - Need to create new stream
-

2. Types of Streams

2.1 Based on Data Type

Stream Type	Description	Example
Stream<T>	Generic stream for any object type	<code>Stream<String>, Stream<Employee></code>
IntStream	Stream of primitive int values	<code>IntStream.of(1,2,3)</code>
LongStream	Stream of primitive long values	<code>LongStream.range(1,10)</code>
DoubleStream	Stream of primitive double values	<code>DoubleStream.of(1.5,2.5)</code>

2.2 Based on Source

Stream Type	Description	When to Use
Sequential Stream	Single-threaded, ordered processing	Default, small to medium datasets
Parallel Stream	Multi-threaded for parallel processing	Large datasets, CPU-intensive operations
Infinite Stream	Unlimited elements using <code>generate()</code> or <code>iterate()</code>	When you need continuous data generation

2.3 Based on Operation Type

Operation Type	Description	Examples
Intermediate	Returns another stream, lazy	<code>filter()</code> , <code>map()</code> , <code>sorted()</code>
Terminal	Produces result or side-effect, eager	<code>forEach()</code> , <code>collect()</code> , <code>reduce()</code>
Stateless	Independent of previous elements	<code>filter()</code> , <code>map()</code>
Stateful	Depends on previous elements	<code>sorted()</code> , <code>distinct()</code>

3. Complete List of Stream Methods

3.1 Intermediate Operations (Lazy)

Filtering

Method	Description	Example
<code>filter(Predicate)</code>	Filters elements based on condition	<code>stream.filter(x -> x > 5)</code>
<code>distinct()</code>	Removes duplicates	<code>stream.distinct()</code>
<code>limit(n)</code>	Truncates to first n elements	<code>stream.limit(10)</code>
<code>skip(n)</code>	Skips first n elements	<code>stream.skip(5)</code>
<code>takeWhile(Predicate)</code>	Takes while condition is true (Java 9+)	<code>stream.takeWhile(x -> x < 10)</code>
<code>dropWhile(Predicate)</code>	Drops while condition is true (Java 9+)	<code>stream.dropWhile(x -> x < 5)</code>

Mapping

Method	Description	Example
<code>map(Function)</code>	Transforms each element	<code>stream.map(x -> x*2)</code>
<code>mapToInt(ToIntFunction)</code>	Maps to IntStream	<code>stream.mapToInt(Employee::getSalary)</code>
<code>mapToLong(ToLongFunction)</code>	Maps to LongStream	<code>stream.mapToLong(x -> x)</code>
<code>mapToDouble(ToDoubleFunction)</code>	Maps to DoubleStream	<code>stream.mapToDouble(x -> x*1.5)</code>
<code>flatMap(Function)</code>	Flattens nested collections	<code>stream.flatMap(list::stream)</code>

<code>flatMapToInt(Function)</code>	Flattens to IntStream	<code>stream.flatMapToInt(x -> IntStream.of(x))</code>
<code>flatMapToLong(Function)</code>	Flattens to LongStream	<code>stream.flatMapToLong(x -> LongStream.of(x))</code>
<code>flatMapToDouble(Function)</code>	Flattens to DoubleStream	<code>stream.flatMapToDouble(x -> DoubleStream.of(x))</code>

Sorting

Method	Description	Example
<code>sorted()</code>	Natural order sorting	<code>stream.sorted()</code>
<code>sorted(Comparator)</code>	Custom comparator sorting	<code>stream.sorted((a,b) -> a-b)</code>

Peeking

Method	Description	Example
<code>peek(Consumer)</code>	For debugging, performs action	<code>stream.peek(System.out::println)</code>

3.2 Terminal Operations (Eager)

Reduction

Method	Description	Example
<code>reduce(BinaryOperator)</code>	Reduces to single value	<code>stream.reduce((a,b) -> a+b)</code>
<code>reduce(identity, BinaryOperator)</code>	Reduction with initial value	<code>stream.reduce(0, (a,b) -> a+b)</code>

<code>count()</code>	Returns count of elements	<code>stream.count()</code>
<code>min(Comparator)</code>	Returns minimum element	<code>stream.min(Comparator.naturalOrder())</code>
<code>max(Comparator)</code>	Returns maximum element	<code>stream.max(Comparator.naturalOrder())</code>
<code>sum()</code> (IntStream/LongStream)	Returns sum	<code>intStream.sum()</code>
<code>average()</code> (IntStream/LongStream)	Returns average	<code>intStream.average()</code>

Collection

Method	Description	Example
<code>collect(Collector)</code>	Collects to collection	<code>stream.collect(Collectors.toList())</code>
<code>toList()</code> (Java 16+)	Collects to List	<code>stream.toList()</code>
<code>toArray()</code>	Returns array	<code>stream.toArray()</code>
<code>toArray(IntFunction)</code>	Returns typed array	<code>stream.toArray(Employee[]::new)</code>

Matching

Method	Description	Example
<code>anyMatch(Predicate)</code>	True if any element matches	<code>stream.anyMatch(x -> x > 5)</code>

<code>allMatch(Predicate)</code>	True if all elements match	<code>stream.allMatch(x -> x > 0)</code>
<code>noneMatch(Predicate)</code>	True if no elements match	<code>stream.noneMatch(x -> x < 0)</code>
<code>findFirst()</code>	Returns first element	<code>stream.findFirst()</code>
<code>findAny()</code>	Returns any element	<code>stream.findAny()</code>

Iteration

Method	Description	Example
<code>forEach(Consumer)</code>	Performs action for each	<code>stream.forEach(System.out::println)</code>
<code>forEachOrdered(Consumer)</code>	Maintains order in parallel	<code>stream.forEachOrdered(System.out::println)</code>

3.3 Special Methods for Primitive Streams

Method	Description	Example
<code>range(start, end)</code>	Range of ints (exclusive end)	<code>IntStream.range(1, 10)</code>
<code>rangeClosed(start, end)</code>	Range of ints (inclusive end)	<code>IntStream.rangeClosed(1, 10)</code>
<code>summaryStatistics()</code>	Statistics (sum, avg, min, max)	<code>intStream.summaryStatistics()</code>
<code>boxed()</code>	Converts primitive to boxed	

